Course Name:	**Digital Signal and Image Processing**	**Semester:**	**VI**
Date of Performance:	01 / 04 / 2026	**DIV/ Batch No:**	C-3
Student Name:	Shreyans Tatiya	**Roll No:**	16010123325

Title : Implement following Edge detection operators (Prewitt, Sobel, Robert, and Laplacian).

Objective : To learn and understand different edge detection operators.

Expected Outcome of Experiment :

Course Outcome	Description
CO-4	Design & implement algorithms for digital image enhancement, segmentation & restoration.

Books / Journals / Websites referred :

1. http://www.mathworks.com/support/
2. www.math.mtu.edu/~msgocken/intro/intro.html.
3. R. C.Gonsales R.E.Woods, “Digital Image Processing”, Second edition, Pearson Education
4. S.Jayaraman, S Esakkirajan, T Veerakumar “Digital Image Processing “Mc Graw Hill.
5. S.Sridhar,”Digital Image processing”, oxford university press, 1st edition."

❖ **Pre-Lab Prior concepts :**

Image segmentation can be achieved in two ways,

1. Segmentation based on discontinuities of intensity.
2. Segmentation based on similarities in intensity.

Edge information in an image is found by looking at the relationship a pixel has with its neighbourhoods. If a pixel’s gray-level value is similar to those around it, there is probably not an edge at that point. If a pixel’s has neighbors with widely varying gray levels, it may present an edge point.

Edge Detection Methods :

Many are implemented with convolution mask and based on discrete approximations to differential operators. Differential operations measure the rate of change in the image brightness function.Some operators return orientation information. Other only return information about the existence of an edge at each point.

1) Sobel Operator

The operator consists of a pair of 3×3 convolution kernels as shown in Figure 1. One kernel is simply the other rotated by 90°.

-1	0	+1
-2	0	+2
-1	0	+1

Gx

+1	+2	+1
0	0	0
-1	-2	-1

Gy

These kernels are designed to respond maximally to edges running vertically and horizontally relative to the pixel grid, one kernel for each of the two perpendicular orientations. The kernels can be applied separately to the input image, to produce separate measurements of the gradient component in each orientation (call these Gx and Gy). These can then be combined together to find the absolute magnitude of the gradient at each point and the orientation of that gradient. The gradient magnitude is given by:

$$|G| = \sqrt{Gx^2 + Gy^2}$$

Typically, an approximate magnitude is computed using:

$$|G| = |Gx| + |Gy|$$

which is much faster to compute. The angle of orientation of the edge (relative to the pixel grid) giving rise to the spatial gradient is given by:

$$\theta = \arctan(Gy/Gx)$$

2) Robert's cross operator

The Roberts Cross operator performs a simple, quick to compute, 2-D spatial gradient measurement on an image. Pixel values at each point in the output represent the estimated absolute magnitude of the spatial gradient of the input image at that point. The operator consists of a pair of 2×2 convolution kernels as shown in Figure. One kernel is simply the other rotated by 90°. This is very similar to the Sobel operator.

+1	0
0	-1

Gx

0	+1
-1	0

Gy

These kernels are designed to respond maximally to edges running at 45° to the pixel grid, one kernel for each of the two perpendicular orientations. The kernels can be applied separately to the input image, to produce separate measurements of the gradient component in each orientation (call these Gx and Gy). These can then be combined together to find the absolute magnitude of the gradient at each point and the orientation of that gradient. The gradient magnitude is given by:

$$|G| = \sqrt{Gx^2 + Gy^2}$$

An approximate magnitude is computed using:

$$|G| = |Gx| + |Gy|$$

Which is much faster to compute? The angle of orientation of the edge giving rise to the spatial gradient (relative to the pixel grid orientation) is given by:

$$\theta = \arctan(Gy/Gx) - 3\pi/4$$

3) Prewitt's operator

Prewitt operator is similar to the Sobel operator and is used for detecting vertical and horizontal edges in images.

$$h_1 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} \quad h_3 = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Write Algorithm and MATLAB commands used :

1. Start
2. Read the input image using imread()
3. Convert the image to grayscale using rgb2gray()
4. Apply edge detection techniques:
    - Roberts
    - Sobel
    - Prewitt
    - LoG
    - Canny
5. Store outputs of each method
6. Display results using imshow() and subplot()
7. Stop

MATLAB Commands Used (In-Built Functions)

- imread() → Reads image
- rgb2gray() → Converts RGB to grayscale
- edge() → Performs edge detection
    - 'roberts', 'sobel', 'prewitt', 'log', 'canny'
- subplot() → Divides figure window
- imshow() → Displays image
- title() → Adds title to images
- figure → Creates new figure window

Algorithm (Manual Sobel Edge Detection)

1. Start
2. Read and convert image to grayscale
3. Convert image to double for calculations
4. Define Sobel masks Gxand Gy
5. For each pixel (excluding borders):
    - Extract 3×3 neighborhood
    - Compute gradient in x-direction (gx)
    - Compute gradient in y-direction (gy)
    - Compute magnitude: sqrt{gx^2 + gy^2}
6. Normalize output image
7. Display result using imshow()
8. Stop

MATLAB Commands Used (Manual Sobel)

- double() → Converts image to double
- size() → Gets image dimensions
- zeros() → Initializes output matrix
- sum() → Computes convolution manually
- sqrt() → Gradient magnitude
- uint8() → Converts back for display
- imshow() → Displays output

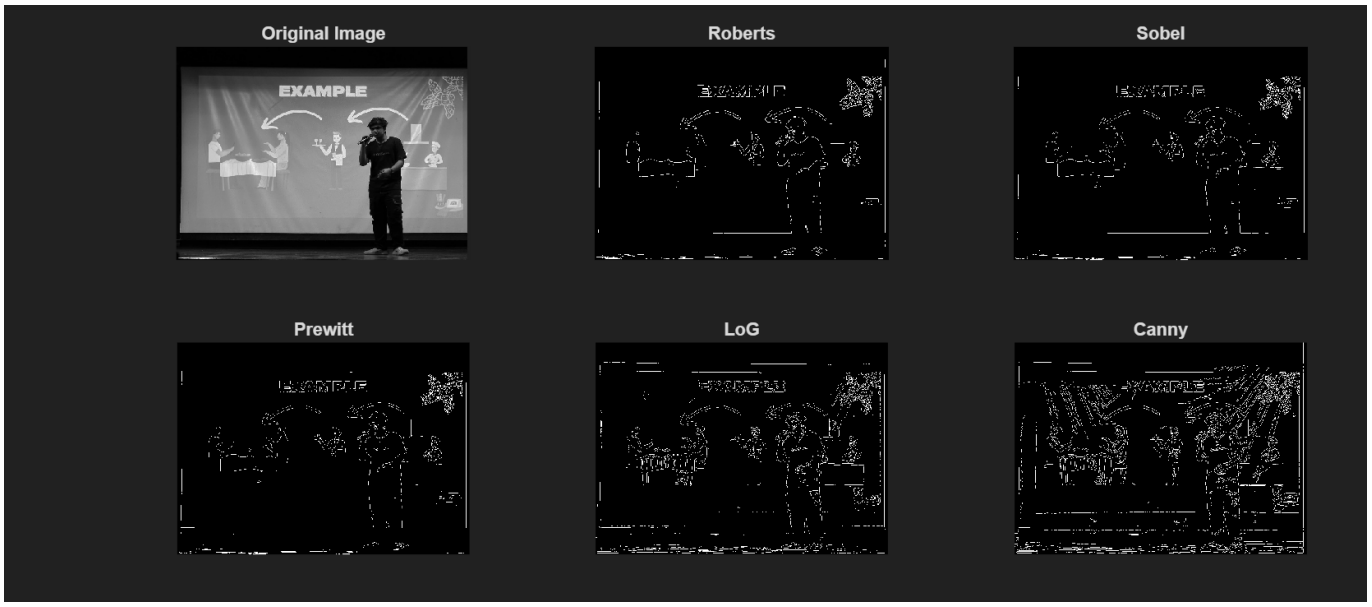
❖ **Implementation Steps with Screenshots :**

MATLAB Code :

(In-Built Function)

```
clear all;
close all;
clc;

% Read your image (replace filename with yours)
a = imread('IMG_6232.jpg');

% Convert to grayscale (important!)
a = rgb2gray(a);

% Apply different edge detectors
b = edge(a, 'roberts');
c = edge(a, 'sobel');
d = edge(a, 'prewitt');
e = edge(a, 'log');
f = edge(a, 'canny');

% Display results
figure;

subplot(2,3,1), imshow(a), title('Original Image');
subplot(2,3,2), imshow(b), title('Roberts');
subplot(2,3,3), imshow(c), title('Sobel');
subplot(2,3,4), imshow(d), title('Prewitt');
subplot(2,3,5), imshow(e), title('LoG');
subplot(2,3,6), imshow(f), title('Canny');
```

Output-

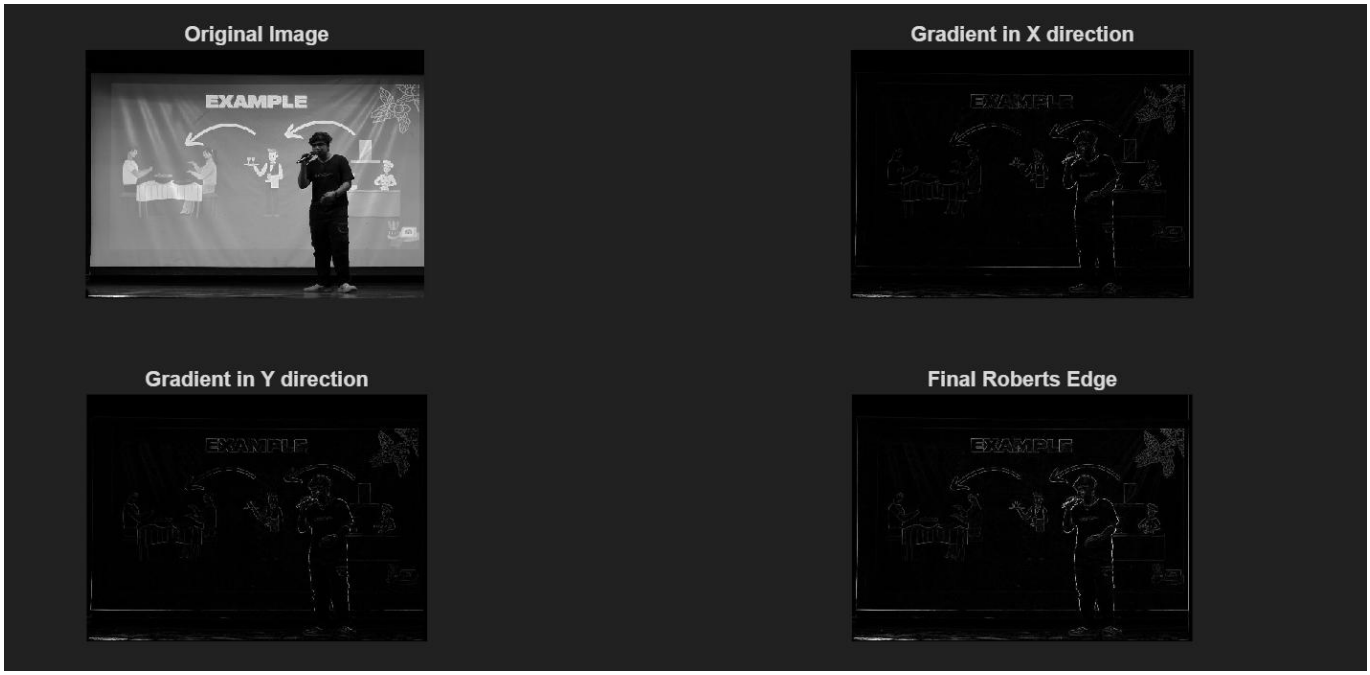


(Manual – Roberts)

```
clear all;
close all;
clc;

% Read image
a = imread('IMG_6232.jpg');
a = rgb2gray(a);
a = double(a);

% Sobel masks
Gx = [-1 0 1; -2 0 2; -1 0 1];
Gy = [-1 -2 -1; 0 0 0; 1 2 1];

[m, n] = size(a);

% Initialize output
edge_img = zeros(m, n);

% Apply convolution manually
for i = 2:m-1
    for j = 2:n-1
        region = a(i-1:i+1, j-1:j+1);

        gx = sum(sum(Gx .* region));
        gy = sum(sum(Gy .* region));
```

```
        edge_img(i,j) = sqrt(gx^2 + gy^2);
    end
end

% Normalize for display
edge_img = uint8(edge_img);

% Show result
imshow(edge_img);
title('Roberts Edge Detection (Manual)');
```

Output-



Conclusion:

In this experiment, various edge detection operators such as Sobel, Prewitt, Roberts, and Laplacian were implemented and analyzed. It was observed that:

- Sobel and Prewitt operators provide good edge detection with noise smoothing.
- Roberts operator is simple but more sensitive to noise.
- Laplacian operator detects fine edges but is highly sensitive to noise and often requires smoothing.

Thus, different operators are suitable for different applications depending on noise levels and edge detail requirements.

❖ **Post-Lab Questions :**

1. **Explain the need of LOG operator.**

- Used for **edge detection**, especially in **noisy images**
- The **Laplacian** alone is very sensitive to noise
- So, we first apply **Gaussian smoothing** to reduce noise
- Then apply **Laplacian** to detect intensity changes

Working:

1. Smooth image (Gaussian)
2. Apply Laplacian
3. Detect edges using **zero crossings**

Advantage:

- Gives **more accurate and clean edges**

2. **Explain the technique of thresholding for segmentation**

- Technique to **segment image into foreground and background**
- Converts grayscale image → **binary image** using threshold **T**

Types:

- **Global Thresholding** → single T for whole image
- **Adaptive Thresholding** → different T for regions
- **Otsu's Method** → automatically finds best T

Advantages:

- Simple and fast
- Useful in **object detection, document processing, medical images**

.

Date: **Signature of faculty In-charge**